

Android in the Living Room: High-Fidelity Cloud Phones on IPTV Edge Devices and Misuse Risks

Lin Jiang

Independent Researcher
pppaass@163.com

Chenke Luo

Tulane University
clu06@tulane.edu

Jiang Ming

Tulane University
jming@tulane.edu

Abstract

In a world increasingly reliant on smartphones, the rising demand for computational power and robust security has given birth to a transformative innovation: the “cloud phone.” This mobile cloud computing platform addresses challenges such as limited computing resources and security risks by deploying mobile operating systems on data center servers via virtualization, allowing users to remotely control these systems. Cloud phones have proven effective in secure workplace environments, game performance optimization, and cost-efficient virtual device testing for developers. However, their potential for misuse has become a growing concern. Cloud phones can be exploited for illegal activities, serving as an infrastructure for large-scale fraudulent operations and facilitating distributed malicious activities.

In this paper, we present novel techniques for achieving high-fidelity Android virtualization on IPTV edge computing devices. Departing from traditional server-based deployment, our approach employs Android container technology to construct realistic virtual phone environments directly on IPTV devices. By customizing device environments and emulating essential hardware components, our developed platform operates covertly without disrupting IPTV functionalities (e.g., Android TV OS) and remains inconspicuous to device owners. Moreover, residential IP addresses inherent to IPTV devices complicate the detection of potential misuse. We experimentally demonstrate the feasibility and effectiveness of our approach, highlighting its significant implications for security, including its potential exploitation for malicious activities. Our findings advocate for strengthened regulatory oversight and advancements in environment detection technologies to mitigate such risks.

1 Introduction

Advances in data-center computing power, high-capacity storage, and the widespread deployment of low-latency, high-bandwidth 5G networks have accelerated the development of *cloud phone* technology, enabling cloud-based services such as remote office solutions and cloud gaming [1]. In a cloud phone architecture, the mobile operating system runs entirely in the cloud and is accessed via dedicated client applications; all computation and data storage occur remotely, decoupling application performance from the capabilities

of end devices [2, 3]. This model provides consistent, high-quality user experiences across heterogeneous clients. Cloud phones strengthen “Bring Your Own Device” security by isolating enterprise data and applications from personal devices [4, 5] and deliver stable performance for mobile gaming across diverse hardware platforms [3, 6–10]. In addition, cloud phones can instantiate large numbers of realistic virtual devices, enabling cost-effective device farms for application development and testing [11–13].

However, cloud phone technology poses significant risks of exploitation by black and gray market activities [14–17]¹. Virtual devices created and controlled in bulk through cloud phone systems can easily bypass traditional network monitoring and defense mechanisms. These virtual devices can be misused for various illegal or malicious activities, such as aggressive marketing campaigns, hacking, gaming fraud, cryptocurrency mining, and app ranking manipulation [18]. The cloud-based deployment of these virtual devices provides a high degree of concealment, making it challenging to monitor and mitigate such malicious actions [19–23]. For instance, machine learning-based mobile malware detection systems [21] and automated security scanners like Bouncer [22] cannot identify malware residing within cloud phone environments when accessed through legitimate cloud phone client applications published in app stores.

In this paper, we explore a novel deployment model for Android container systems by repurposing IPTV edge devices to host high-fidelity virtual phones. Unlike traditional cloud-based virtualization platforms that run on centralized data center servers, this approach leverages continuously active, household-deployed IPTV boxes to instantiate virtual Android environments. These virtual instances operate unobtrusively in the background and are associated with legitimate residential IP addresses, making them both persistent and difficult to distinguish from physical mobile devices. While this architectural strategy improves availability and realism, it also introduces potential security concerns, particularly in detecting covert misuse, managing system integrity, and ensuring responsible deployment. We analyze the risks of such deployments being exploited for malicious online

¹Black and gray markets refer to clandestine or semi-legal digital marketplaces and operations where cybercriminals engage in the exchange of goods, services, and information that facilitate malicious activities.

activities and argue for stronger design-time safeguards and runtime environment verification.

In particular, this study builds upon the Cells framework [24] to construct one or more background virtual Android systems on IPTV edge computing devices. Our platform, named *VPhone*, refines these virtual Android systems through systematic environment customization and hardware-module emulation, creating a high-fidelity mobile runtime that supports the proper installation and execution of real-world mobile apps. In addition, virtual display devices, virtual cameras, and virtual network interfaces are integrated into the virtual Android systems to enable seamless remote access and interaction. An overview of *VPhone* and its environment customization pipeline is provided in Appendix A.

Using this system architecture, our study demonstrates how high-fidelity virtual Android instances can be systematically deployed on edge computing devices. This approach enables the construction of scalable, geographically distributed virtual phone clusters without relying on centralized data center infrastructure. By minimizing hardware and bandwidth requirements, the system significantly reduces the per-instance deployment cost and improves accessibility for broader use cases. However, this same scalability and distribution nature of these deployments makes them harder to trace, increasing the risk of misuse in scenarios such as coordinated network attacks or fraudulent automation.

We conducted a series of comparative experiments to evaluate the performance, effectiveness, and device customization capabilities of *VPhone*, benchmarking it against prominent Android virtualization platforms such as Cuttlefish [25], vSoC [26], and WayDroid [27]. Additionally, we deployed *VPhone* clusters on both data center servers and IPTV boxes to further assess its effectiveness. These clusters were utilized to log into and interact with ten of the most popular user-interaction applications, performing routine operations for analysis. The results demonstrate that the *VPhone*-based cloud phone cluster outperforms Cuttlefish-based cluster in bypassing app runtime environment anomaly detection. The *VPhone*-based cluster exhibited superior emulation capabilities, enhanced concealment, and a higher success rate in executing application operations.

Using the *cybercriminal value chain model* [20] for analysis, cloud phone clusters constructed on IPTV edge computing devices serve as critical infrastructure within the cyberattack ecosystem, offering substantial utility for malicious purposes. However, a significant challenge lies in the need for collusion with IPTV manufacturers to achieve large-scale deployment and implementation. One potential scenario involves pre-installing such cloud phones on edge computing devices imported into underdeveloped regions. Due to slower technological advancements and weaker regulatory oversight in these areas, issues with such devices may go unnoticed.

Furthermore, this type of highly concealed cloud phones is difficult for device owners to detect. For instance, in July 2024, the German government announced the removal of certain imported telecommunications equipment [28], likely due to the significant security risks associated with these devices and the inherent challenges in regulating their use. This example underscores the potential threats posed by compromised edge computing devices. In summary, our paper makes the following contributions:

- We design and implement a practical approach for deploying high-fidelity Android container systems on commodity IPTV edge devices, providing a scalable and cost-effective alternative to traditional data center-based virtualization.
- We conduct an extensive experimental evaluation comparing IPTV-based cloud phone clusters with state-of-the-art Android virtualization platforms, showing higher compatibility with commercial apps, better concealment, and robust support for long-running services on resource-constrained edge hardware.
- We highlight potential misuse scenarios stemming from large-scale deployment of edge-hosted virtual Android instances. Our findings underscore regulatory challenges and the need for improved environment detection mechanisms in securing future edge-based software systems.

Open Science Policy To foster more research on the *VPhone* platform, we have released a prototype of *VPhone* at [Zenodo](#).

2 Background and Related Work

This section reviews the technical foundations of our work. We first summarize cloud phone architectures and IPTV hardware/software stacks as our edge deployment target, then discuss Android container-based virtualization and environment customization for high-fidelity emulation. Together, these works motivate and constrain the design of *VPhone*.

2.1 Cloud Phone Technology

Advancements Cloud phones, a mobile cloud computing platform, revolutionize mobile app usage by running entirely on cloud servers instead of local devices. Users access cloud phone systems through desktop programs or mobile applications, interacting with apps on the cloud phone just as they would with locally installed apps, achieving a seamless user experience. This decoupled architecture eliminates hardware constraints by offloading computation and data storage to the cloud, enabling high-performance execution [2]. Performance no longer relies on the computational capabilities of the end device. Even low-end devices can leverage the advanced computational power of cloud phones, including high-performance CPUs, GPUs, and AI accelerators [29].

Cloud phones also enhance data security by centralizing data and applications in the cloud, reducing risks of leaks and theft. They address “Bring Your Own Device” concerns by safeguarding enterprise data and isolating applications, preventing unauthorized access to personal information [4, 5]. Additionally, cloud phones significantly lower the cost of mobile app testing by eliminating the need to procure diverse devices for compatibility testing. By emulating various app environments and generating virtual devices in bulk, cloud phones enable the construction of cost-efficient virtual device testing farms [11–13].

Virtualization Support Cloud phone systems can be deployed on data center servers or edge computing devices [30]. While servers provide substantial computational power and storage, edge devices offer limited resources, making lightweight virtualization technologies like containers more suitable [31, 32]. Such container solutions create low-resource virtual instances, improving efficiency in mobile computing and application offloading [33–35]. Conversely, servers are better suited for more resource-intensive virtualization approaches, such as virtual machine (VM). Several advanced platforms exemplify cloud phone virtualization: Anbox [2] and Waydroid [27] are currently the most popular Android container-based cloud phone solutions. Similar to Docker technology, they are deployed on servers in data centers, where the host operating systems are traditional Linux distributions such as Ubuntu and CentOS. These host systems only support essential functions such as computing, storage, and networking. The virtual Android instances created by Anbox and Waydroid are optimized and lightweight. Besides, both vSoC [26] and Cuttlefish [25] are virtual machine solutions optimized on top of VMM and KVM. vSoC [26] is an efficient virtual mobile SoC platform targeting heterogeneous hardware, enabling high-fidelity virtual hardware module sharing through a unified shared virtual memory framework. Cuttlefish [25], Google’s next-generation configurable virtual Android device, supports both remote deployment on third-party cloud platforms (e.g., Google Cloud Engine) and local execution on x86 or ARM64 machines. It delivers high-fidelity behavior replication for system and application developers without relying on physical hardware.

High-Fidelity Emulation Many apps, especially those in sensitive domains such as finance, detect abnormal system environments [36–39]. Checks include rooting status, presence of plugins, device tampering, activation of security defenses, and whether the device is an emulator or a non-mainstream brand. Abnormalities can result in restricted app functionality or login bans. Cloud phones must, therefore, emulate realistic mobile environments, including hardware modules like SIM cards, GPS, and WiFi networks, to pass

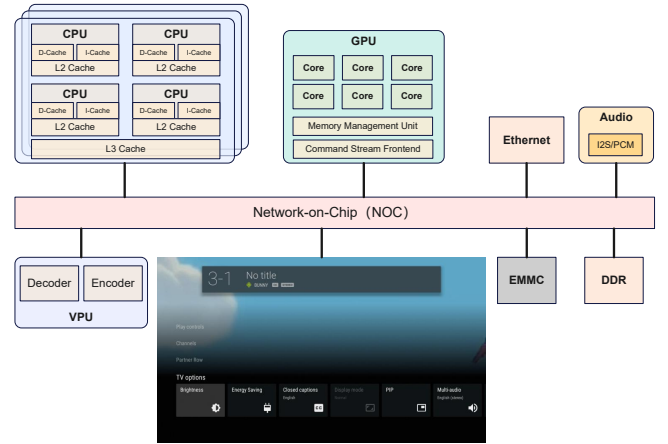


Figure 1: Hardware architecture of an IPTV set-top box.

such anomaly detection. High-fidelity emulation allows apps to function seamlessly within virtual environments.

Multimedia Device Requirements Cloud phones are also commonly used for secure office scenarios, where video conferencing apps are critical. These apps must interact with local hardware, such as cameras and audio devices, to capture real-time data and enable communication through the cloud. Apps like WeChat Work and DingTalk may even require facial recognition for login, necessitating remote access to local cameras and microphones for recording and authentication.

2.2 Internet Protocol Television (IPTV)

IPTV is a technology for delivering television content over internet protocols, distinguishing it from traditional satellite or cable television by providing programs and video-on-demand (VOD) through internet networks. Key features of IPTV include VOD, live television, karaoke, cloud gaming, and multi-screen interaction. Its advantages lie in its rich content offerings and high-quality visuals. Users require a stable broadband connection, an IPTV set-top box, and a smart TV to access IPTV services. Figure 1 illustrates the hardware architecture of an IPTV set-top box. These devices feature audio-video processing modules, such as VPU, to encode and decode television content, enhancing transmission efficiency and quality. Additionally, they are equipped with high-performance GPU for real-time rendering of video content displayed on TV screens. Large-capacity storage modules are included to cache video data, alongside audio and network interfaces for connecting voice devices and external networks. Prominent IPTV set-top box manufacturers include HiSilicon [40], Broadcom [41], Amlogic [42], and Sigma Designs [43].

The software layer of IPTV set-top boxes typically runs the Android TV OS alongside custom applications for live streaming, video playback, music, and more. Android TV

OS [44] is specifically designed for television devices, optimized for large screens to ensure ease of operation and control. It supports a wide range of applications, including video streaming services and gaming, and enables 4K and HDR video playback, delivering an exceptional viewing experience [45]. Whether an IPTV box powers off when the television is turned off depends on its design and settings. Most mainstream boxes remain active to enable background updates and continuous network functionality [40–43].

2.3 Android Container-based Virtualization

The Cells framework [24] is a lightweight Android virtualization solution capable of creating multiple Android containers on the same smartphone, enabling simultaneous operation in an isolated and secure manner. Cells introduces a conceptual model of foreground and background container systems, where only one container is in the foreground while others operate in the background. It implements a novel device namespace mechanism and data proxying methods integrated with Linux namespaces and cgroups, allowing efficient hardware multiplexing across multiple containers with minimal performance loss. The containers in Cells support full Android functionality, including system interfaces, power management, and telephony, but rely heavily on mobile hardware platforms.

Building upon Cells, AutoVP [46] applies an enhanced version in smart vehicles to create intelligent cockpit systems that meet functional safety requirements. AutoVP uses modular hardware to provide significant cost advantages and ease of deployment in scenarios requiring multiple cooperating Android containers. Its hardware platform (e.g., two low-cost chips) is customized for collaborative use cases.

Anbox [2] and Waydroid [27] are open-source projects designed to run Android containers on desktop OSs like Ubuntu. By leveraging the hardware acceleration capabilities of servers or personal computers, they enable Android apps to run smoothly in desktop environments. However, the Android containers built with Anbox and Waydroid provide only limited functionalities, such as virtual display, virtual touch, and virtual audio services, which are suitable for cloud gaming scenarios but lack virtualization support for critical devices like camera, battery, SIM card, and WiFi networking.

2.4 Challenges in Implementing Android Containers on IPTV Devices

Android container can also be applied to IPTV devices to construct edge computing resource pools. When building Android containers on IPTV devices, it is essential to avoid interfering with the primary TV OS. Hardware virtualization cannot rely on shared services, data proxying, or mutex-based hardware multiplexing methods between the TV OS

and Android containers, as this would disrupt core IPTV hardware functions like display and input modules.

A possible solution involves enabling remote invocation of API functions from another machine, a technique referred to as *API remoting* [47]. In this approach, system API calls are intercepted within the virtual system and relayed via a specialized RPC protocol to a remote client module. The client module parses these data packets, invokes the corresponding hardware module functionality, and asynchronously returns the results to the virtual system. API remoting is widely used in GPU virtualization to enable the multiplexing of OpenGL, OpenCL, and CUDA interfaces in virtual machines [10, 48].

Another challenge in building virtual phone systems on IPTV devices is hardware platform differences. Unlike mobile devices, IPTV hardware lacks modules such as cameras, batteries, GPS, sensors, and SIM cards. Virtual Android systems must emulate these absent hardware components internally. Virtual hardware modules can be constructed using semi-virtualization techniques like Virtio [50], commonly employed in virtual machine solutions such as QEMU, Cuttlefish, and vSoC. Virtio emulates hardware at the kernel level, providing emulated hardware functionalities to the upper system layers. However, Android containers on IPTV devices cannot directly adopt Virtio because both the Android container and host OS share a single kernel, making Virtio unsuitable for the host kernel. Instead, Android containers can leverage Virtio’s principles by constructing virtual hardware modules at the Android HAL (Hardware Abstraction Layer), the system layer closest to the kernel. This approach allows effective hardware emulation tailored for Android containers running on IPTV devices.

2.5 Customizing Virtual System Environments

The system environment of physical or virtual devices can be customized through altering hardware parameters and program conditional logic. These customizations enable applications to receive inconsistent results when querying system APIs. Such tailored environments can serve specific purposes, including simulating attack scenarios [51], analyzing malware [52], and reducing testing costs [12]. Mobile apps often perform environment checks at launch to detect and block non-compliant devices such as emulators or rooted phones. For instance, TikTok restricts features such as liking or commenting if it detects an emulator during login, and banking apps deny access on rooted devices [36, 37]. By customizing the system environment, virtual devices and emulators can deceive such detection mechanisms, making applications treat these systems as legitimate mobile devices.

Three previous works have explored customizing virtual system environments. VPDroid [51] and its follow-up work,

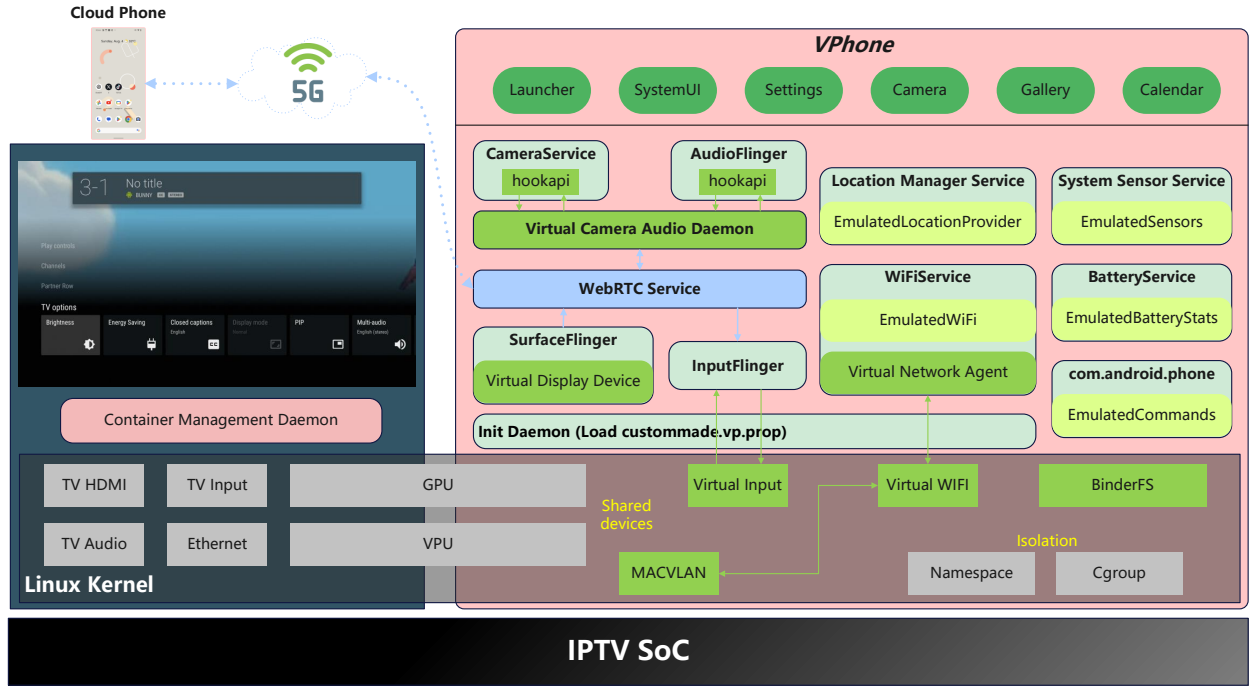


Figure 2: VPhone is built upon the IPTV SoC architecture, using WebRTC [49] for remote access to virtual phones. This approach enables efficient real-time interaction with the virtual devices over the network.

VPBox [52], customize device attributes, such as CPU model, Android ID, and phone number, on real mobile devices to emulate the environment of another smartphone. They achieve this by modifying three software layers: Android system properties, user space, and kernel space. In addition, they use container isolation mechanisms to ensure that apps within the emulated environment cannot detect these modifications. DeviceFarmer [12] leverages virtual devices to emulate high-fidelity system environments, reducing the development and testing costs of applications. The framework involves selecting emulators and client operating systems, installing device vendor-specific services, resolving graphics resource compatibility issues, supporting vendor-specific backend policies, and developing dynamic binary patching techniques.

While VPDroid and VPBox focus on limited environment customization for security analysis, and DeviceFarmer targets testing in non-production environments using Android emulators, these methods fall short in emulating realistic mobile environments on non-mobile platforms. For IPTV devices to emulate a “trusted” and “realistic” mobile environment capable of bypassing production-level app environment checks, comprehensive and non-interfering customization is required. This involves tailoring system software, modifying hardware SoC details, and emulating mobile hardware modules. Additionally, these environments can be continually adjusted to meet application-specific detection mechanisms.

3 Motivation and Overview

Emerging Security Risks Recently, the low cost and multi-instance capabilities of cloud phones have been exploited by malicious actors. They use cloud phones to control large numbers of fake accounts, engaging in illegal activities such as fraudulent marketing, online scams, and malicious attacks. Examples include fake transaction boosting, account farming, and DDoS attacks [53–55]. The inherent concealment and anonymity of cloud phones make tracking and combating such illegal activities particularly challenging.

Our work is motivated by a real-world security incident that one of our co-authors helped investigate. In a South-east Asian region, users noticed their IPTV boxes remained unusually warm overnight, despite being turned off. Suspecting hidden background activities, security professionals examined the devices using ADB and standard diagnostic tools. They found that, in addition to running the Android TV OS, the IPTV boxes were executing numerous Android processes resembling an independent Android system. This led to the hypothesis that the IPTV box also hosted a separate virtualized Android system.

VPhone Overview To facilitate the study of emerging security risks associated with cloud phones, we explore Android container and device customization technologies to construct high-fidelity virtual mobile systems, termed *VPhone*, on IPTV boxes. VPhone replicates nearly all core functionalities of an

Android smartphone and successfully passes environment detection mechanisms employed by mainstream mobile apps. Furthermore, VPhone is deployable not only on IPTV boxes but also on similar edge computing platforms, supporting the creation of concealed and distributed cloud phone clusters.

System Architecture Figure 2 illustrates VPhone’s architecture. VPhone partially leverages mechanisms from Cells [24], such as foreground-background system separation and kernel-level device virtualization. VPhone employs container technology to create one or more virtual Android systems on top of IPTV SoC. These virtual systems operate exclusively in the background, while the TV OS remains in the foreground. The virtual Android system features a smartphone-sized virtual display. It supports mobile app installation, offers smartphone-like operational methods, and includes an independent network subsystem for remote internet access.

Please note that VPhone’s virtual Android systems do not share the display or multimedia subsystems with the TV OS. Instead, they are equipped with independent virtual screens, virtual cameras, and virtual audio devices, ensuring seamless and non-disruptive operation alongside the TV OS. Graphics rendering and video encoding tasks share GPU and VPU with the TV OS. Due to the highly parallelized architecture of GPU and VPU kernel drivers, these resources can be efficiently shared without disrupting the foreground TV OS. This architecture allows virtual Android systems to run covertly on IPTV devices for specific tasks.

Environment Customization Environment customization aims to ensure that the virtual phone replicates the behavior of a real smartphone, allowing apps to function normally without dependence on physical mobile hardware. In addition to virtualizing specific hardware modules (e.g., camera and audio), VPhone achieves a high-fidelity virtual mobile environment through two key approaches: (1) customization of virtual environment, such as SoC hardware specs, manufacturer details, and Android system info; and (2) emulation of essential hardware modules, including SIM cards, GPS, WiFi, power management, and sensors.

4 Android Containers on IPTV Devices

VPhone utilizes Android container technology based on the Cells framework [24] to create independently operating Android container computing units on IPTV boxes. Equipped with virtual display devices, virtual input devices, virtual cameras, and virtual network subsystems, VPhone operates independently of the TV OS, ensuring seamless functionality without interfering with the TV OS. In the following subsections, we detail our approaches to addressing the unique challenges identified in §2.4.

4.1 API Remoting

To ensure that VPhone operates without disrupting the TV OS, and given that IPTV boxes lack complex smartphone hardware modules such as telephony, GPS, sensors, and power management systems, VPhone does not utilize the device virtualization methods provided by Cells (e.g., data proxying and mutex-based mechanisms) to multiplex IPTV’s physical hardware. Instead, VPhone adopts API remoting [47] to virtualize *display devices*, *input devices*, *cameras*, and *audio devices*. API remoting involves embedding hook functions within the device framework service functions. These hook functions intercept device calls and forward the requests to a remote client, which uses its hardware modules to perform the corresponding operations. In addition, this method requires deploying corresponding functional modules on the cloud phone client to execute instructions for the virtual hardware modules of the cloud phone. Command and result data are managed using queue-based caching, with all virtual module calls being asynchronous. Adaptive data flow transmission is achieved through the caching queue.

VirtualCamera API Remoting Next, we use camera virtualization as an example to explain the API remoting mechanism we implemented. Figure 3 (1~6) illustrates the process flow of the VirtualCamera API remoting.

- (1) **Hooking Functions:** The CameraService in the cloud phone instance is modified by embedding hook functions into methods like “openCamera”, which package the function identifier, parameters, and return type into a command data block.
- (2) **Inter-process Communication:** CameraService uses the Binder mechanism to communicate with the VirtualCameraAudioDaemon module, pushing the command data block into a caching queue.
- (3) **Remote Command Retrieval:** The ControlModule on the cloud phone’s remote client retrieves command data blocks from the VirtualCameraAudioDaemon cache queue via an external network.
- (4) **Command Execution:** The client-side ControlModule parses the data block to extract the command identifier, parameters, and return type. Based on this information, it invokes the corresponding functional module and stores the result in the caching queue.
- (5) **Result Transmission:** The ControlModule notifies the VirtualCameraAudioDaemon module in the cloud phone to fetch the result data. VirtualCameraAudioDaemon retrieves the result and caches it in the queue.
- (6) **Result Handling:** The CameraService service retrieves the result data from the cache queue and submits the invocation result to the upper-layer application using an event mechanism.

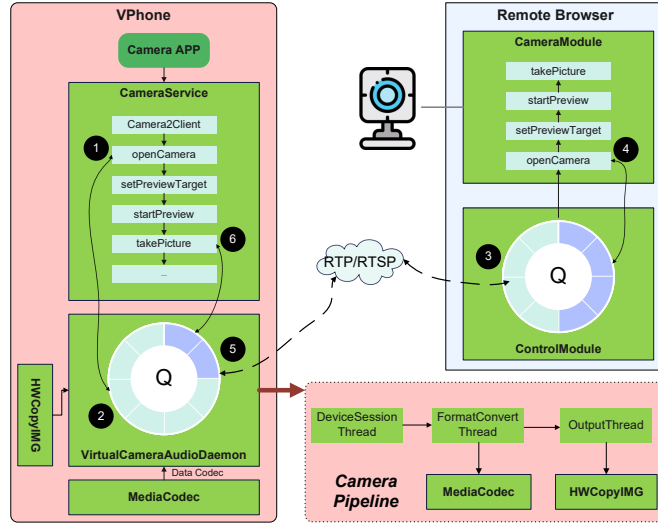


Figure 3: The workflow of API remote invocation.

The implemented virtual camera supports RTP (Real-Time Transport Protocol) and RTSP (Real-Time Streaming Protocol). The remote acquisition module deploys RTP and RTSP servers to handle audio/video capture and encoding, while the virtual Android system hosts audio/video decoding and rendering modules (RTPDecoder and RTSPDecoder). These decoding and rendering tasks are asynchronous and operate in separate threads. The virtual camera effectively supports remote functionalities such as QR code scanning, facial recognition, and live streaming. Additionally, it enables the playback of video files within the virtual Android system.

Smooth Virtual Camera Rendering To mitigate CPU workload and maintain smoothness, we introduce a kernel-level image frame transfer module based on a DMA co-processor. As shown in Figure 3’s “HWCopYIMG,” this module maintains mappings among the physical addresses, virtual addresses, and memory block IDs of image frames, and leverages the DMA co-processor to copy data between memory blocks, thereby avoiding CPU involvement in virtual camera data transfer and reducing CPU workload. Experimental results (see Figure 8) show that, with this module handling image movement, the virtual camera maintains smooth visual output even in the presence of non-negligible end-to-end latency.

4.2 Optimizing Multimedia Transmission

The use of API remoting for virtual display, camera, and audio devices involves the transmission of large volumes of video and audio data. To reduce bandwidth usage, improve transmission efficiency, and lower storage requirements while enabling real-time communication, we also employ audio and video encoding/decoding techniques. For instance, the virtual display module encodes and compresses

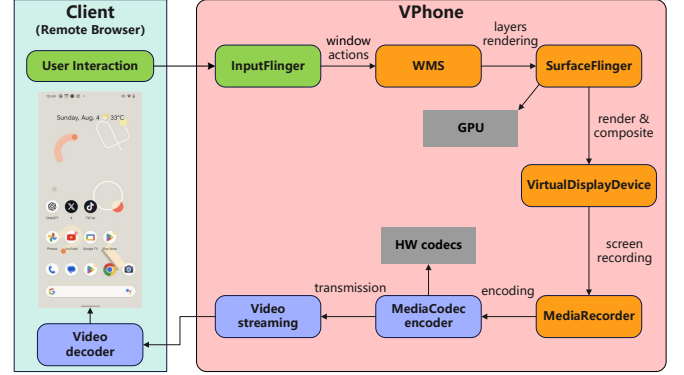


Figure 4: Remote display workflow in VPhone.

the cloud phone’s desktop content in real time, transmitting the encoded video data to the cloud phone client. The ControlModule on the client side decodes and decompresses the received video data, then renders the desktop display using the display application.

IPTV boxes, designed for large-screen televisions, are equipped with powerful hardware acceleration units for audio and video encoding/decoding. These units enhance transmission efficiency and improve the quality of audio and video outputs. Typically, such hardware accelerators feature highly parallel architectures, allowing them to quickly process large volumes of multimedia data and execute complex audio and video algorithms simultaneously. VPhone’s Android containers leverage the audio and video encoding/decoding hardware accelerators on IPTV devices to optimize the efficiency of API remoting. This integration enhances the performance of the cloud phone’s virtual hardware modules.

Remote Display Workflow As shown in Figure 4, users interact with VPhone through a remote client, with input events transmitted over the network to VPhone’s InputFlinger module. The WindowManagerService (WMS) processes the input events to update display content and layers, which are subsequently submitted to SurfaceFlinger for rendering. Using the GPU, SurfaceFlinger renders and composites these layers into a virtual screen buffer, referred to as VirtualDisplayDevice. The recording service, utilizing MediaRecorder, captures the composite surface in the VirtualDisplayDevice and encodes it into a video stream via the MediaCodec hardware encoding module. The remote client receives this video stream, decodes it using a negotiated decoding algorithm, and displays the graphical content. Please note that the GPU rendering framework on IPTV devices executes layer rendering tasks at the process level. To accommodate multiple containers, the framework requires adaptation for resource multiplexing. Once adapted, virtual systems created using Linux container technology can share GPU computational resources with other systems on the device. This resource sharing is designed to ensure that the

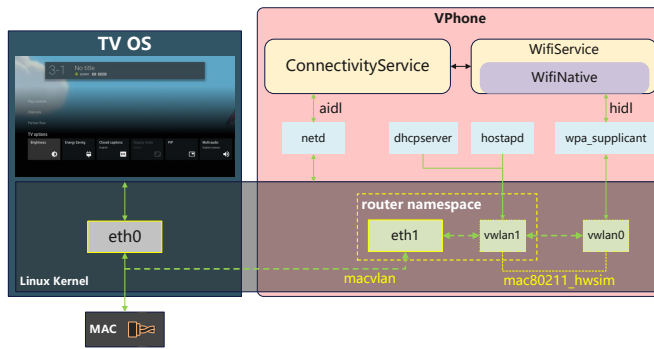


Figure 5: Constructing virtual WiFi network.

utilization of GPU modules remains interference-free between systems.

4.3 WiFi Virtualization

Figure 5 illustrates the construction of a virtual WiFi network using MACVLAN [56] and MAC80211_HWSIM [57]. IPTV boxes typically rely on Ethernet and lack native wireless capabilities, so VPhone first employs MACVLAN to segment a physical network interface into multiple virtual interfaces (e.g., `eth0` and `eth1`), each with its own IP and MAC address, allowing communication both among virtual interfaces and with external networks. Then, VPhone employs MAC80211_HWSIM, a virtual WiFi hardware simulation driver in the Linux kernel, to create two interconnected virtual WiFi nodes: `vwlan0` and `vwlan1`.

The `vlan0` node is configured in Station mode and managed by `wpa_supplicant` to establish an Android active network, while the `vlan1` node is set to Access Point mode and configured by `hostapd` to simulate a WiFi hotspot. To ensure network isolation from the virtual Android system, a namespace named *router* is created, and `eth1` and `vlan1` are moved into it. NAT (Network Address Translation) is then configured on these interfaces, enabling their traffic to reach the host and internet. Cellular (SIM) data traffic can likewise be mapped onto this virtual network. To the best of our knowledge, this is the first work to combine MACVLAN and MAC80211_HWSIM to provide a full virtual network stack within an Android container.

4.4 Virtualization of Binder and Input

Binder Virtualization Binder is a critical inter-process communication (IPC) mechanism in Android systems. To ensure complete isolation between the TV OS and VPhone, Binder virtualization does not reuse the Binder device from the TV OS. Instead, it involves creating “vbinder” nodes and implementing a service-sharing mechanism. Binder virtualization leverages the BinderFS framework introduced in Android 11 [58]. BinderFS, a Linux kernel-based filesystem,

is specifically designed for managing Binder devices by organizing multiple Binder instances within a single filesystem, thereby simplifying the management of Binder drivers. By using BinderFS, each container can mount its own independent BinderFS filesystem, ensuring complete isolation of Binder drivers between containers. This capability is crucial in environments requiring multiple isolated Binder drivers, such as Android containers.

Input Virtualization EVDEV [59] is a Linux kernel subsystem designed to handle input devices, particularly those adhering to the EVDEV interface standard. The EVDEV device interface facilitates the transmission of input events, such as keyboard presses, mouse movements, and touchscreen interactions, between user-space programs and the kernel, providing a unified event-handling mechanism for various input devices in Linux systems. For input virtualization, we adapt the EVDEV device driver and use Linux *kernel* namespace to isolate `evdev_client` objects. This ensures VPhone input events remain fully isolated from those of the TV OS or other Android containers, maintaining independent and interference-free input functionality.

5 VPhone's Environment Customization

VPhone achieves a high-fidelity virtual environment in three ways: basic environment customization, emulation of selected mobile hardware modules, and virtualization of specific hardware components. Detailed descriptions of these methods are provided in Appendix B.

The basic environment includes key parameters such as SoC hardware information, manufacturer details, Android version information, and Android system properties. These parameters can be customized and then are stored as key-value pairs in a configuration file,² which is imported by the root process during system startup. This design enables dynamic access to these properties at runtime.

The emulated hardware modules include SIM cards, GPS, WiFi networks, sensors, and power management. These modules are emulated using static HAL (Hardware Abstraction Layer) components at the framework layer. These static HAL components provide basic functionalities, such as hardware information queries and simple configuration operations, ensuring that the entire virtual Android system, including applications, perceives the presence of hardware modules. They also ensure that system API calls to hardware functionalities do not result in errors. Since these static HAL components do not involve kernel operations, they won't interfere with the host TV system or other virtual systems.

The virtualized hardware modules include display, input, camera, and audio, implemented through API remoting. As we presented in §4, our method enables VPhone to interact

²The name of the configuration file is `custommade.prop`.

with these hardware functionalities efficiently and without impacting other systems running on the same device.

5.1 Customization by Device Emulation

Next, we use SIM card emulation as an example to demonstrate how static HAL components are utilized within the framework layer of hardware modules, enabling the virtual system to query hardware information. As shown in Figure 6, the RILJ (Radio Interface Layer Java) module in the Android phone service traditionally interacts with the SIM card HAL service (RILD) to utilize SIM card functionalities. However, since IPTV devices lack physical SIM cards, VPhone constructs a static HAL component called “EmulatedCommands” within the Android container. The RILJ module interacts with “EmulatedCommands” to retrieve SIM card information, such as phone numbers, service providers, IMEI (International Mobile Equipment Identity), and MEID (Mobile Equipment Identifier). The “EmulatedCommands” component provides all necessary functional interfaces to the phone service, preventing system errors during phone-related operations.

Similarly, WiFi, GPS, sensor, and battery modules each have corresponding static HAL components added to their respective Android system services. These components handle hardware data queries and parameter configurations. By avoiding user-space to kernel-space interactions, inter-process or inter-service communications, and the complex encapsulation and parsing of data protocols, static HAL-based device emulation simplifies data processing, improving system stability and response speed. Our experimental results in §6.2 demonstrate that the use of static HAL components enables the virtual Android system to emulate physical hardware modules accurately, supporting the creation of a realistic and stable virtual smartphone environment.

5.2 The Limits of Virtual Realism

In an escalating battle between application security and testing efficiency, some apps now scrutinize their host environments with unprecedented precision, probing for signs of tampering or emulation. Certain applications perform rigorous checks on the system environment, evaluating factors such as root access capabilities, whether security mechanisms like SELinux are enforced, and whether the Android build version is a “user” version with all security features enabled. Emulators and virtual systems, frequently used for application debugging and testing, benefit from disabling these security measures to improve testing efficiency. However, applications detecting the absence of such security features often classify the system as an emulator, restricting or disabling specific functionalities. Furthermore, some applications assess whether screen dimensions align with typical smartphone displays or whether the current display

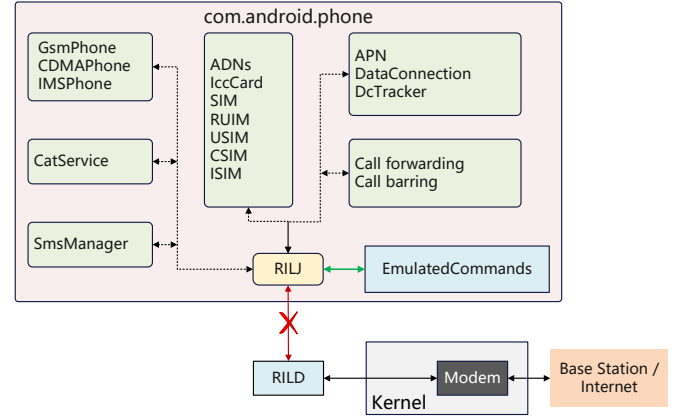


Figure 6: The structure of telephony service and emulated static HAL component.

layer is rendered on a physical screen. When the display layer is not tied to a physical screen, security-critical layers, such as login or payment interfaces, may fail to render.

As a result, environment customization is a dynamic process that necessitates iterative analysis based on experimental results. This approach helps identify how apps detect environments and pinpoint customized environment aspects that do not meet app requirements. According to the Law of Leaky Abstractions [60], any minor modification to a large-scale OS with numerous users will inevitably be detected and potentially exploited by certain applications through implicit dependencies. For example, specific APIs may initialize hardware in real environments, causing slower performance. Apps might leverage such “slow” APIs for synchronization tasks. However, in virtual environments, where these APIs exhibit minimal response delays, these synchronization tasks may fail. Therefore, a customized virtual environment can *closely approximate a real environment but can never fully replicate it*.

6 Evaluation

In this section, we first evaluate the performance of VPhone and compare it with state-of-the-art virtualization platforms. Next, we conduct experimental assessments using both benchmark tests and real-world applications to measure VPhone’s effectiveness, focusing on its fidelity, integrity, and app compatibility. The evaluation is conducted using Android 13 and Linux kernel 5.10 across three different devices, each with the following hardware configurations:

- (1) **Real Phone:** CPU: 4 cores at 2.0GHz, Memory: 8GB DDR4, Storage: 64GB eMMC, Network: WIFI6, GPU: PowerVR SGX544.
- (2) **IPTV Device:** CPU: 4 cores at 2.0GHz, Memory: 8GB DDR4, Storage: 64GB eMMC, Network: Supports 10M / 100M / 1000Mbps adaptive, GPU: ARM G52 2EE.

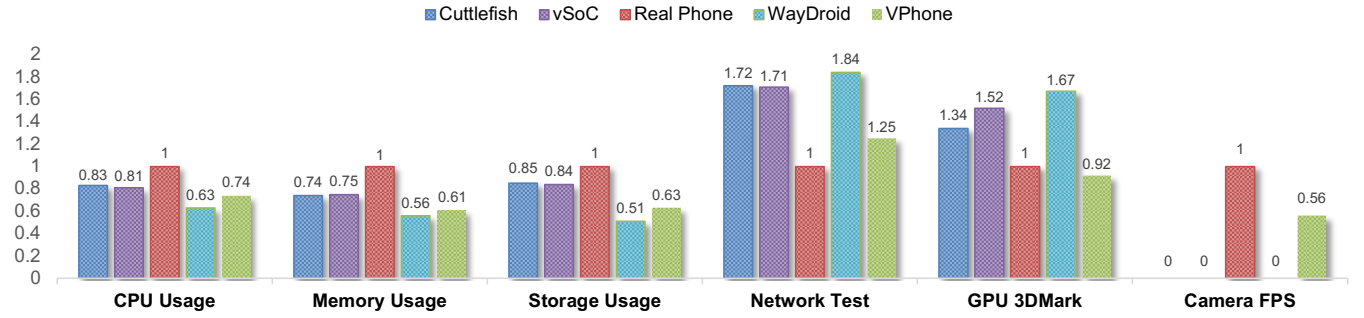


Figure 7: Normalized performance comparison results. For CPU Usage, Memory Usage, and Storage Usage, a lower score is better. For Network Test, GPU 3DMark, and Camera FPS, a higher score is better.

- (3) **ARM Server:** CPU: 64 cores at 2.0GHz, Memory: 256GB DDR4, Local Disk: 800GB SSD, Network Card: 210GE, GPU: 3WX5100.

6.1 Performance

We begin by running AnTuTu performance benchmark [61] on a real phone running Android 13, using the results as the baseline. The benchmark data includes CPU usage, memory usage, storage usage, network performance, GPU 3DMark, and camera FPS. Next, we deploy VPhone on an IPTV device and run a custom-configured Android 13 virtual container system within VPhone. For comparison, we evaluate three prominent Android virtualization platforms—Cuttlefish [25], vSoC [26], and WayDroid [27]—all running Android 13 on an ARM server. We then execute the same benchmarks across these four virtual environments and compare their performance. It is important to note that directly comparing the results of the IPTV device with those of the emulators on the ARM server is not entirely fair, as the IPTV device, being an embedded system, has a significant performance gap compared to the ARM server. Therefore, we use the real phone’s test data as the baseline and normalize the results from the IPTV device and ARM server before making the comparison.

We did not deploy three peer virtualization platforms on IPTV devices for the following reasons: Cuttlefish [25] and vSoC [26] primarily rely on x86 or ARM64 server architectures. Their underlying KVM/VMM virtualization requires substantial CPU performance and memory resources, exceeding the limited capabilities of embedded SoCs commonly found in IPTV devices. Additionally, WayDroid [27] depends on a complete Linux desktop environment, whereas IPTV devices typically utilize a streamlined Android TV OS that lacks comprehensive support for a GNU/Linux runtime environment. The normalized results are presented in Figure 7.

In terms of CPU, memory, and storage usage, both Android emulators (including Cuttlefish and vSoC) and containers (including VPhone and WayDroid) consume fewer resources. This is because both emulators and containers are virtualized environments, where many features are limited. For example,

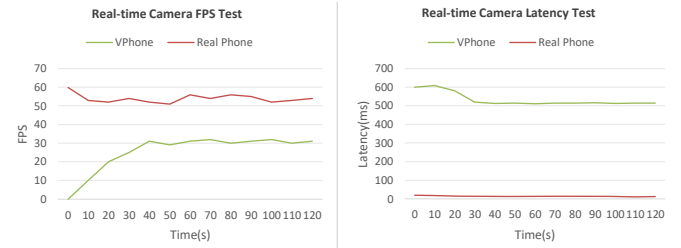


Figure 8: Camera module performance test results: smoothness (measured by FPS) vs. latency.

virtual devices lack phone-specific functionality, and the corresponding software components are absent from the virtual system. Additionally, Android containers leverage lightweight virtualization technology, which consumes fewer computational resources compared to Android emulators.

Regarding GPU performance, the ARM server is equipped with a high-performance graphics card, resulting in a higher GPU 3DMark test score for the Android emulator compared to the real phone device. In contrast, VPhone achieves a GPU 3DMark score similar to that of the phone device. This is because VPhone shares the GPU with the IPTV device, and the GPU performance of the IPTV is comparable to that of the phone device’s GPU.

In terms of networking, the ARM server and IPTV are connected via Ethernet, while the phone device uses a Wi-Fi connection. As a result, both network latency and uplink/downlink speeds are significantly better on Ethernet compared to Wi-Fi, even though the virtual devices utilize network virtualization technologies. Regarding camera FPS (Frames Per Second), two Android emulators and WayDroid lack real-time virtual camera functionality, resulting in a test score of 0. In contrast, VPhone’s virtual camera implements real-time video capture through API remote invocation, video encoding/decoding, and network transmission. Experimental results indicate that VPhone’s virtual camera achieves roughly half the frame rate of a real mobile device. Furthermore, we observed that WayDroid, as an Android container system optimized for cloud gaming and server deployment, generally outperforms VPhone in testing metrics.

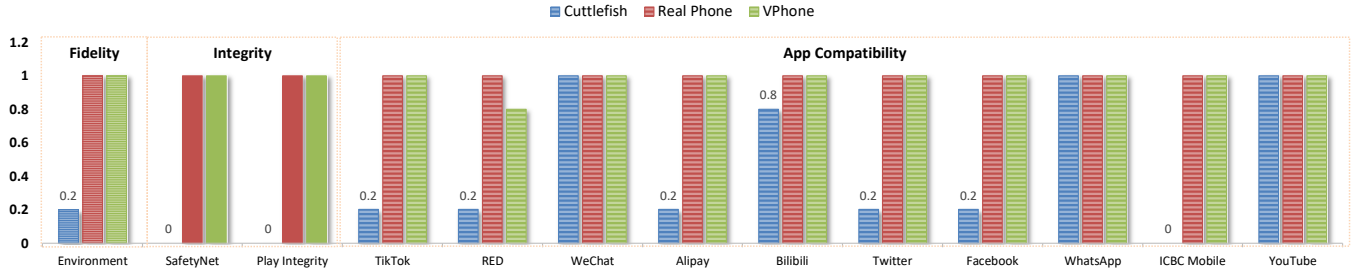


Figure 9: Fidelity, integrity, and app compatibility evaluation results.

Virtual Camera Performance Since VPhone is the only solution among the four virtualization approaches that implements virtual camera functionality, we conduct additional experiments to evaluate its performance. We deploy the VPhone camera remote client on a real phone and ensure that both the IPTV device and the real phone are connected to the same local area network, thereby eliminating any network-related performance variability. For the experiments, we use the real-time transmission protocol (RTP), with the video capture module set to H.264 encoding, YUV420 image format, a bitrate of 0.5x60x1080x1920 (bps), a frame rate of 60 fps, and a keyframe interval of 5 seconds. We then simultaneously activate the camera on both VPhone and the real phone for a set duration, recording FPS and latency data every 10 seconds. The results are presented in Figure 8.

Experimental results indicate that VPhone’s camera exhibits a moderate display latency of approximately 520 milliseconds, with a frame rate of about 30 frames per second. These results align with our expectations, as the virtual camera implementation in VPhone involves multiple processing stages, including remote video capture, encoding, network transmission, decoding, and rendering—each stage contributing to latency and computational overhead.

Latency vs. Smoothness Regarding the noticeable virtual camera latency (500–600 ms) shown in Figure 8, we clarify that latency and smoothness represent distinct dimensions of camera performance. Latency refers to the delay between image capture and its display, primarily resulting from network transmission and video encoding/decoding processes. In contrast, smoothness, typically measured by frames per second (FPS), indicates how fluidly the images are displayed. Thanks to our DMA-based frame transfer mechanism, our virtual camera consistently achieves ~30 FPS, which is generally sufficient for typical mobile applications.

6.2 Effectiveness

Next, we evaluate VPhone’s fidelity, integrity, and app compatibility. Among the platforms tested, vSoC and WayDroid, due to their limited Android functionality, were unable to complete the experiments. Only Cuttlefish completed all

tests. Thus, we compared the effectiveness of VPhone against Cuttlefish for the subsequent evaluation.

Fidelity First, we run the AnTuTu real device detection benchmark [61] on a real phone to collect device, software, hardware, and module information. Next, we use VPhone and Cuttlefish to emulate the real phone environment and run the same benchmark to gather equivalent data. We then assign weights to each information type and compare the results from VPhone and Cuttlefish with the real phone’s information. The nine information types, as well as detailed test parameters and their corresponding scores, are shown in Table A2 of Appendix. Among these, software information is weighted at 0.2 points because of its greater complexity compared to other types of information. Based on our empirical study, in consideration of user experience, apps typically utilize less than 80% of device environment parameters within each type listed in Table A2 for the environment detection. Therefore, we define that a score is awarded for each type if the similarity exceeds 80%; otherwise, no score is given.

Integrity Next, we run the SafetyNet [62] and Play Integrity [63] applications to assess the system’s integrity. SafetyNet and Play Integrity are security services provided by Google that use cryptographic techniques, such as signature verification and hash summaries, to determine whether the device passes Google’s device certification. These services help protect Android apps from fraud, cheating, and abuse. A score of 1 is awarded for passing the basic integrity checks of both services; otherwise, the score is 0. Play Integrity is the latest replacement for SafetyNet.

App Compatibility Finally, we conduct a compatibility evaluation by testing 10 representative apps. These ten apps were selected due to their widespread popularity and large user bases. Additionally, some of them feature the most advanced environment detection mechanisms (e.g., ICBC Mobile Banking). These apps incorporate strong environment detection mechanisms (e.g., to identify rooted, emulated, or tampered devices), but they are also designed to preserve user experience by avoiding false positives, i.e., they typically do not block legitimate users who happen to use less common or customized mobile devices. The app compatibility test involves the following five operations: installation,

login, routine operations, maintaining login for 30 minutes, and maintaining login for one day. For each operation, if the application functions without crashing and is not flagged as running in a virtual environment, it earns 0.2 points; otherwise, no points are awarded.

The experimental results for the three tests are shown in Figure 9. In the **Fidelity** test, Cuttlefish scored only 0.2 points in the test of software information, while VPhone scored 1.0 point, indicating that the environment simulated by VPhone closely mirrors a real-world environment. Notably, VPhone required the most effort to maintain the consistency of the software information, as this involves a large number of software components. For **Integrity**, VPhone can pass the basic integrity checks of SafetyNet and Play Integrity, indicating the system in the test environment was not modified by root access or any unauthorized changes. In contrast, Cuttlefish was correctly identified as emulators by these services.

For **App Compatibility**, VPhone successfully completed the majority of the test items, with the only failure occurring during the one-day login test for REDNote. We speculate that this failure was caused by insufficient hardware resources on the IPTV device. To validate this hypothesis, we deployed VPhone on the ARM server and repeated all compatibility tests, and all tests including REDNote’s one-day login test, were successfully completed. In contrast, Cuttlefish failed all tests for ICBC and encountered multiple failures in tests involving TikTok, REDNote, Alipay, Twitter, Facebook, and Bilibili. It only managed to complete the installation step for TikTok, REDNote, Alipay, Twitter, and Facebook, while failing the login, routine operations, 30-minute login persistence, and one-day login persistence tests. Additionally, on Cuttlefish, the Bilibili app failed to display the payment page properly, resulting in a failure for the routine operations test.

Upon further investigation, we found that banking and stock trading apps (e.g., ICBC Mobile Banking), when performing emulator detection, verify whether the device is an emulator, rooted, or an authentic branded smartphone. In addition, these applications examine whether the current system utilizes a physical display, whether the file system is read-only, and request GPS location information at the time of login. TikTok conducts checks to determine if the device is an emulator, rooted, or a genuine branded smartphone. It also monitors the frequency of specific user behaviors, such as liking and commenting. Alipay and Bilibili verify whether the device is an emulator or rooted, and assess if the system is operating with a physical display. Twitter and Facebook merely check whether the device is an emulator or rooted. Other applications, such as instant messaging services (e.g., WeChat and WhatsApp), do not impose stringent requirements regarding the device environment.

In summary, with its fully customized system, VPhone demonstrates strong fidelity, integrity, and app compatibility

across a range of test scenarios. Our experimental results further indicate that VPhone’s environment closely approximates that of a physical smartphone.

7 Discussion & Conclusion

In this final section, we discuss key practical considerations arising from our work, including the ethical considerations of our study, methods for detecting the presence of VPhone, and the broader implications of deploying VPhone on future edge devices, before offering our concluding remarks.

Ethics Considerations We carefully addressed ethical considerations by conducting all experiments within controlled environments, avoiding interactions with real-world systems, and responsibly disclosing our findings to relevant regulatory authorities and internet application vendors. For an extensive discussion of the ethical considerations and our mitigation strategies, please refer to Appendix C.

Detecting the Presence of VPhone We summarize three potential methods for detecting the presence of VPhone: (1) network bandwidth monitoring, as continuous transmission of virtual Android interface images significantly increases network usage; (2) storage usage analysis, since multiple virtual Android systems running simultaneously cause noticeable growth in occupied storage space; and (3) temperature monitoring, as higher workloads from concurrent Android systems cause increased thermal output of hardware components. Interested readers can refer to Appendix D for an in-depth discussion of these detection mechanisms and their practical implications.

Future Edge Deployments As the first complete system design for constructing virtual mobile systems on edge devices, VPhone breaks new ground where no prior work has explored heterogeneous embedded platforms. With rising edge computing power and increasingly interconnected environments, such virtualization may soon be commonplace—for example, in smart homes where miniature servers host multiple virtual systems, each securely mapped to a household device’s display for seamless and isolated operation.

Conclusion This work investigates the use of Android container technologies to enable high-fidelity virtualization of mobile environments on IPTV devices. We presented a practical approach for repurposing commodity IPTV hardware to host virtual Android systems, which support realistic mobile app execution and operate covertly alongside standard IPTV functions. Our evaluation highlights the technical viability and effectiveness of this approach, while also exposing its potential misuse for large-scale, distributed deployment of virtual devices in unauthorized or malicious contexts. Our study provides new insights into emerging mobile-cloud architectures at the edge, underscoring the need for improved monitoring, detection, and regulatory mechanisms.

References

- [1] Niroshinie Fernando, Seng W. Loke, and Wenny Rahayu. Mobile Cloud Computing: A Survey. *Future Generation Computer Systems*, 29(1):84–106, 2013.
- [2] Anbox Cloud. Scalable Android in the Cloud. <https://anbox-cloud.io/>, [Online].
- [3] Qifan Yang, Zhenhua Li, Yunhao Liu, Hai Long, Yuanchao Huang, Jiaming He, Tianyin Xu, and Ennan Zhai. Mobile Gaming on Personal Computers with Direct Android Emulation. In *Proceedings of The 25th Annual International Conference on Mobile Computing and Networking (MobiCom '19)*, pages 1–15, 2019.
- [4] Peiyu Liu, Yi Chen, Lirong Fu, Mingyuan Yan, and Zonghui Wang. cMobiDesk: A Lightweight Solution for Android Desktop Virtualization. In *Proceedings of the 7th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA '22)*, pages 234–239, 2022.
- [5] Kui Su, Peiyu Liu, Liang Gu, Wenzhi Chen, Kai Hwang, and Zhibin Yu. vMobiDesk: Desktop Virtualization for Mobile Operating Systems. *IEEE Access*, 8, 2020.
- [6] Microsoft. Xbox Cloud Gaming. <https://www.xbox.com/en-US/cloud-gaming>, [Online].
- [7] Ryan Shea, Jiangchuan Liu, Edith C-H Ngai, and Yong Cui. Cloud Gaming: Architecture and Performance. *IEEE Network*, 27(4):16–21, 2013.
- [8] Chun-Ying Huang, Cheng-Hsin Hsu, Yu-Chun Chang, and Kuan-Ta Chen. GamingAnywhere: An Open Cloud Gaming System. In *Proceedings of the 4th ACM Multimedia Systems Conference (MMSys '13)*, pages 36–47, 2013.
- [9] Yang Li, Jiaxing Qiu, Hongyi Wang, Zhenhua Li, Feng Qian, Jing Yang, Hao Lin, Yuhao Liu, Bo Xiao, Xiaokang Qin, and Tianyi Xu. Dissecting and Streamlining the Interactive Loop of Mobile Cloud Gaming. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*, 2025.
- [10] Di Gao, Hao Lin, Zhenhua Li, Chengen Huang, Yunhao Liu, Feng Qian, Liangyi Gong, and Tianyin Xu. Trinity: High-Performance Mobile Emulation through Graphics Projection. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*, pages 285–301, 2022.
- [11] Amazon. AWS Device Farm. <https://aws.amazon.com/cn/device-farm/>, [Online].
- [12] Hao Lin, Jiaxing Qiu, Hongyi Wang, Zhenhua Li, Liangyi Gong, Di Gao, Yunhao Liu, Feng Qian, Zhao Zhang, Ping Yang, and Tianyin Xu. Virtual Device Farms for Mobile App Testing at Scale: A Pursuit for Fidelity, Efficiency, and Accessibility. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking (MobiCom '23)*, pages 1–17, 2023.
- [13] Google. Google Firebase Test Lab. <https://firebase.google.com/products/test-lab>, [Online].
- [14] Lillian Ablon, Martin C Libicki, and Andrea A Golay. *Markets for Cybercrime Tools and Stolen Data: Hackers' Bazaar*. Rand Corporation, 2014.
- [15] Cormac Herley and Dinei Florêncio. Nobody Sells Gold for the Price of Silver: Dishonesty, Uncertainty and the Underground Economy. In *Economics of Information Security and Privacy*, 2010.
- [16] Nicolas Christin. Traveling the Silk Road: A Measurement Analysis of a Large Anonymous Online Marketplace. In *Proceedings of the 22nd international conference on World Wide Web (WWW '13)*, 2013.
- [17] Kurt Thomas, Damon McCoy, Chris Grier, Alek Kolcz, and Vern Paxson. Trafficking Fraudulent Accounts: The Role of the Underground Market in Twitter Spam and Abuse. In *Proceedings of the 22nd USENIX Conference on Security (USENIX Security '13)*, 2013.
- [18] Muhammad Ikram, Lucky Onwuzurike, Shehroze Farooqi, Emiliano De Cristofaro, Arik Friedman, Guillaume Jourjon, Mohammed Ali Kaafar, and M. Zubair Shafiq. Measuring, Characterizing, and Detecting Facebook Like Farms. *ACM Transactions on Privacy and Security (TOPS)*, 20(4), September 2017.
- [19] Guangxuan Chen, Anan Huang, Bo Hu, and Guangxiao Chen. Analysis of Internet Black Market in New Types of Cyber-Related Crime-Taking Personal Information Transaction as an Example. In *SHS Web of Conferences*, volume 163, 2023.
- [20] Keman Huang, Michael Siegel, and Stuart Madnick. Systematically Understanding the Cyber Attack Business: A Survey. *ACM Computing Surveys (CSUR)*, 51(4):1–36, 2018.
- [21] Liangyi Gong, Zhenhua Li, Feng Qian, Zifan Zhang, Qi Alfred Chen, Zhiyun Qian, Hao Lin, and Yunhao Liu. Experiences of Landing Machine Learning onto Market-Scale Mobile Malware Detection. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys '20)*, pages 1–14, 2020.
- [22] Jon Oberheide and Charlie Miller. Dissecting the Android Bouncer. *SummerCon2012, New York*, 95:110, 2012.
- [23] Yuxuan Yan, Zhenhua Li, Qi Alfred Chen, Christo Wilson, Tianyin Xu, Ennan Zhai, Yong Li, and Yunhao Liu. Understanding and Detecting Overlay-Based Android Malware at Market Scales. In *Proceedings of the 17th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '19)*, pages 168–179, 2019.
- [24] Jeremy Andrus, Christoffer Dall, Alexander Van't Hof, Oren Laadan, and Jason Nieh. Cells: A Virtual Mobile Smartphone Architecture. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP '11)*, pages 173–187, 2011.
- [25] Google. Cuttlefish: Android Device Virtualization Platform. <https://source.android.google.cn/docs/devices/cuttlefish>, [Online].
- [26] Jiaxing Qiu, Zijie Zhou, Yang Li, Zhenhua Li, Feng Qian, Hao Lin, Di Gao, Haitao Su, Xin Miao, Yunhao Liu, et al. vSoC: Efficient Virtual System-on-Chip on Heterogeneous Hardware. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*, pages 558–573, 2024.
- [27] Waydroid Team. Waydroid: Android in a Linux Container. <https://waydro.id/>, [Online].
- [28] Voice of America. Germany in Deal to Cut Huawei's Role in 5G Wireless Network. <https://tinyurl.com/ys73x7d5>, [Online].
- [29] Zhipeng Fu, Jun Zhou, and Wanpeng Xu. A GPU-Enabled Mobile Telemedicine Training System for Graphic Rendering. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking (MobiCom '22)*, pages 877–879, 2022.
- [30] Gopika Premsankar, Mario Di Francesco, and Tarik Taleb. Edge Computing for the Internet of Things: A Case Study. *IEEE Internet of Things*, 5(2):1275–1284, 2018.
- [31] Roberto Morabito, Vittorio Cozzolino, Aaron Yi Ding, Nicklas Beijar, and Jorg Ott. Consolidate IoT Edge Computing with Lightweight Virtualization. *IEEE Network*, 32(1):102–111, 2018.
- [32] Roberto Morabito. Virtualization on Internet of Things Edge Devices with Container Technologies: A Performance Evaluation. *IEEE Access*, 5:8835–8850, 2017.
- [33] Red Hat. Containers vs VMs. <https://www.redhat.com/en/topics/containers/containers-vs-vm>s, [Online].
- [34] Wes Felter, Alexandre Ferreira, Ram Rajamony, and Juan Rubio. An Updated Performance Comparison of Virtual Machines and Linux Containers. In *Proceedings of the 2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS '15)*, pages 171–172, 2015.
- [35] Roberto Morabito, Jimmy Kjällman, and Miika Komu. Hypervisors vs. Lightweight Virtualization: A Performance Comparison. In *Proceedings of the 2015 IEEE International Conference on Cloud Engineering (IC2E '15)*, pages 386–393, 2015.

- [36] Ioannis Gasparis, Zhiyun Qian, Chengyu Song, and Srikanth V Krishnamurthy. Detecting Android Root Exploits by Learning from Root Providers. In *Proceedings of the 26th USENIX Security Symposium (USENIX Security '17)*, pages 1129–1144, 2017.
- [37] Muhammad Ibrahim, Abdullah Imran, and Antonio Bianchi. SafetyNOT: On the Usage of the SafetyNet Attestation API in Android. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '21)*, pages 150–162, 2021.
- [38] Google. SafetyNet API. <https://developer.android.google.cn/privacy-and-security/safetynet>, [Online].
- [39] Google. Play Integrity API. <https://developer.android.com/google/play/integrity>, [Online].
- [40] HiSilicon. STB Smart Media. <https://www.hisilicon.com/en/products/smart-media/STB/Hi3798MV310>, [Online].
- [41] Broadcom. BCM7218X Set-Top Box. <https://www.broadcom.com/products/broadband/set-top-box/bcm7218x>, [Online].
- [42] Amlogic. Quad-core Smart Audio SoC for Far-Field Voice Applications. <https://www.amlogic.com/#Products/407/index.html>, [Online].
- [43] Sigma Designs. IPTV Sigma Home. <https://iptvsigma.com/home>, [Online].
- [44] Android TV. Google's operating system for smart TVs and streaming devices. <https://www.android.com/tv/>, [Online].
- [45] Android Open Source Project. TV Input Framework. <https://source.android.google.cn/docs/devices/tv>, [Online].
- [46] Lin Jiang, Feiyu Zhang, and Jiang Ming. Towards Intelligent Automobile Cockpit via A New Container Architecture. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*, pages 205–219, 2024.
- [47] Virtual Open Systems. API Remoting Turnkey Solution for Virtualizing Hardware Resources. <http://www.virtualopensystems.com/en/products/api-remoting/>, [Online].
- [48] José Duato, Antonio J Pena, Federico Silla, Rafael Mayo, and Enrique S Quintana-Orti. rCUDA: Reducing the Number of GPU-Based Accelerators in High Performance Clusters. In *Proceedings of the 2010 International Conference on High Performance Computing & Simulation (HPCS '10)*, pages 224–231, 2010.
- [49] WebRTC. Real-time Communication for the Web. <https://webrtc.github.io/webrtc-org/native-code/android>, [Online].
- [50] Linux Kernel Documentation Team. Virtio: The Linux Kernel Virtual I/O Mechanism. <https://docs.kernel.org/driver-api/virtio/virtio.html>, [Online].
- [51] Wenna Song, Jiang Ming, Lin Jiang, Han Yan, Yi Xiang, Yuan Chen, Jianming Fu, and Guojun Peng. App's Auto-Login Function Security Testing via Android OS-Level Virtualization. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*, pages 1683–1694, 2021.
- [52] Wenna Song, Jiang Ming, Lin Jiang, Yi Xiang, Xuanchen Pan, Jianming Fu, and Guojun Peng. Towards Transparent and Stealthy Android OS Sandboxing via Customizable Container-Based Virtualization. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*, pages 2858–2874, 2021.
- [53] Abisola Tanzako. Phone Farms and Ad Fraud. <https://clickpatrol.com/phone-farms/>, April 2023.
- [54] Smart Megwai. Phone Farms Exposed: The Sneaky Tech Behind Fake Likes, Clicks, and More. <https://tinyurl.com/3tyau5jd>, June 2024.
- [55] Neeraj Soni. Phone Farms: A Closer Look. <https://www.cyberpeace.org/resources/blogs/phone-farms-a-closer-look>, July 2024.
- [56] Hangbin Liu. Introduction to Linux Interfaces for Virtual Networking. <https://tinyurl.com/3fb3zb53>, October 2018.
- [57] Jouni Malinen. mac80211_hwsim - software simulator of 802.11 radio(s) for mac80211. https://docs.kernel.org/netwoorking/mac80211_hwsim/mac80211_hwsim.html, 2008.
- [58] The Linux Kernel Archives. The Android BinderFS Filesystem. <https://docs.kernel.org/admin-guide/binderfs.html>, [Online].
- [59] The Linux Kernel Archives. Linux Input Subsystem userspace API. <https://docs.kernel.org/input/input.html>, [Online].
- [60] Joel Spolsky. The Law of Leaky Abstractions. <https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/>, November 2002.
- [61] AnTuTu Benchmark. A software benchmarking tool commonly used to benchmark smartphones and other devices. <https://www.antutu.com/en/doc/index.htm>, [Online].
- [62] Google. Protect Against Security Threats with SafetyNet. <https://developer.android.com/privacy-and-security/safetynet>, [Online].
- [63] Google. Overview of the Play Integrity API. <https://developer.android.com/google/play/integrity/overview>, [Online].

Appendix

A VPhone Overview

Our developed platform, named *VPhone*, customizes the virtual Android systems through software techniques and emulates a mobile operating environment, enabling the proper installation and functioning of mobile apps. Additionally, virtual display devices, virtual cameras, and virtual networks are integrated into the virtual Android systems to facilitate remote access and operation. Figure A1 illustrates an overview of VPhone and its environment customization.

B Three Methods for Customizing a High-Fidelity Virtual Environment

VPhone achieves a high-fidelity virtual environment in three ways: basic environment customization, emulation of selected mobile hardware modules, and virtualization of specific hardware components. Table A1 outlines three methods for customizing a high-fidelity virtual environment.

C Ethics Considerations

This research involves constructing covert cloud phone instances on IPTV boxes to analyze and understand the associated security risks. While these scenarios involve potentially negative outcomes, such as the misuse of this research by malicious actors (which would require collusion with IPTV manufacturers for large-scale deployment), careful measures were taken to mitigate risks and maintain ethical compliance. First, all experiments were conducted in a controlled environment, ensuring no interaction with or disruption to real-world systems or services. Additionally, no production systems or user data were accessed, guaranteeing that no harm was inflicted on deployed systems or individuals. Second, to minimize unintended consequences, all findings were responsibly disclosed to relevant regulatory authorities and internet application vendors, ensuring that stakeholders could enhance the regulation of edge computing devices and advance application environment detection technologies.

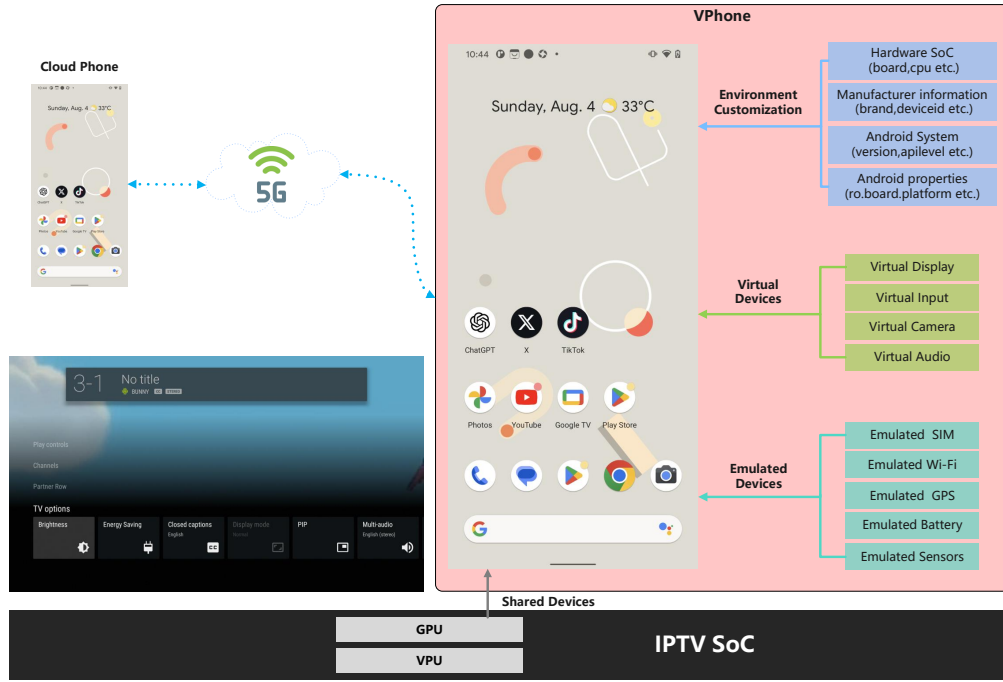


Figure A1: VPhone overview and environment customization.

D Detecting the Presence of VPhone

Next, we discuss several potential methods to detect the presence of VPhone, aiming to provide guidance for manufacturers to update their firmware accordingly in the future. The first detection method involves monitoring the network traffic generated by the device, specifically examining unusual traffic patterns in IPTV equipment, such as frequent interactions with remote servers. VPhone requires real-time transmission of Android interface images. Even when employing compression techniques, continuous transmission of low-resolution interface images still occurs, resulting in noticeable additional network bandwidth consumption. Consequently, a significant increase in network bandwidth usage can serve as an indicator of VPhone presence.

The second detection method entails manually mounting the IPTV storage device onto a computer and examining the storage utilization within its system and user partitions. If the IPTV device has both Android TV OS and VPhone

pre-installed, the occupied space in these partitions will significantly increase. This increase is particularly noticeable when attackers install numerous applications on the VPhone, resulting in evident growth in the user partition's occupied storage space.

A third detection approach involves monitoring the physical temperature of the device. If an IPTV device simultaneously runs multiple Android systems, its CPU, GPU, VPU, and other hardware modules will operate under substantially heavier workloads. Under these conditions, the device's thermal management may exceed expected operating ranges, causing components to experience elevated temperatures compared to normal operation. One practical method is to power on and connect the device to the network while disabling its standard IPTV functionalities (such as television services). If the device continues to exhibit elevated operating temperatures after a certain period, this strongly suggests the existence of additional unintended workloads beyond its expected IPTV functionality.

Table A1: Constructing a customized virtual environment in three ways.

Mechanisms	Categories	Detailed Customization Information
Customize Basic Environment	Basic Manufacturer Information	Device name, brand name, device model, manufacturer name, platform name, device ID, device fingerprint, hardware serial number.
	SoC Hardware Basic Information	Hardware platform name, processor model, processor architecture, processor frequency, manufacturing process, GPU renderer name, GPU manufacturer name, GPU version, GPU frequency.
	Software System Basic Information	Operating system version and name, API version, system version number, build time, device version number, security patch level, JAVA Runtime version, JAVA Virtual Machine, kernel architecture, kernel version, OpenGL version, OpenSSL version.
	Android System Properties	Including <code>ro.build.type</code> , <code>ro.vendor.build.id</code> , <code>ro.boot.bootreason</code> , and others.
Emulate Hardware Modules	SIM	Static HAL Component: EmulatedCommands
	GPS	Static HAL Component: EmulatedLocationProvider
	WiFi	Static HAL Component: EmulatedWiFi
	Sensor	Static HAL Component: EmulatedSensors
	Battery	Static HAL Component: EmulatedBatteryStats
Virtualize Hardware Modules	Display	Hooked System Service: VirtualDisplayDevice Corresponding Client Service: RemoteDisplay
	Input	Hooked System Service: InputFlinger Corresponding Client Service: RemoteInput
	Camera	Hooked System Service: CameraService Corresponding Client Service: RemoteCamera
	Audio	Hooked System Service: AudioFlinger Corresponding Client Service: RemoteAudio

Table A2: Detailed fidelity test parameters and their corresponding scores. Among the nine information types, software information is weighted at 0.2 points because of its greater complexity compared to other types of information.

Information Type	Detailed Environment Parameters	Score
Device Information	Device name, brand name, model, manufacturer, platform name, device ID, device fingerprint, hardware serial number, system properties.	0.1
Software Information	Operating system version and name, API level, system version number, build time, device version number, security patch level, Java Runtime version, Java Virtual Machine, kernel architecture, kernel version, OpenGL version, OpenSSL version.	0.2
Hardware Information	Hardware platform name, processor model, architecture, frequency, manufacturing process, GPU renderer name, GPU manufacturer, GPU version, GPU frequency.	0.1
GPS Module	Longitude, latitude, altitude, speed, speed accuracy, heading, heading accuracy.	0.1
Network Module	Wi-Fi network status, BSS, SSID, BSSID, interface name, 5G support, connection speed, frequency, SIM network name, network type, network status, phone number, IMEI number.	0.1
Battery Module	Charging/discharging status, battery capacity, current charge level, current, voltage, power, temperature, technology type.	0.1
Display Module	Screen type, width, height, size, physical dimensions, refresh rate, HDR support, resolution, screen DPI, logical density, scaling density.	0.1
Camera Module	Resolution, focal length, aperture, thumbnail size, sensor size, target FPS range, white balance mode, exposure range, focus range, flashlight.	0.1
Sensor Module	Accelerometer, magnetometer, orientation sensor, gyroscope, light sensor, pressure sensor, proximity sensor, gravity sensor, linear acceleration sensor, rotation vector sensor, step detector, pedometer, device orientation sensor (manufacturer name).	0.1